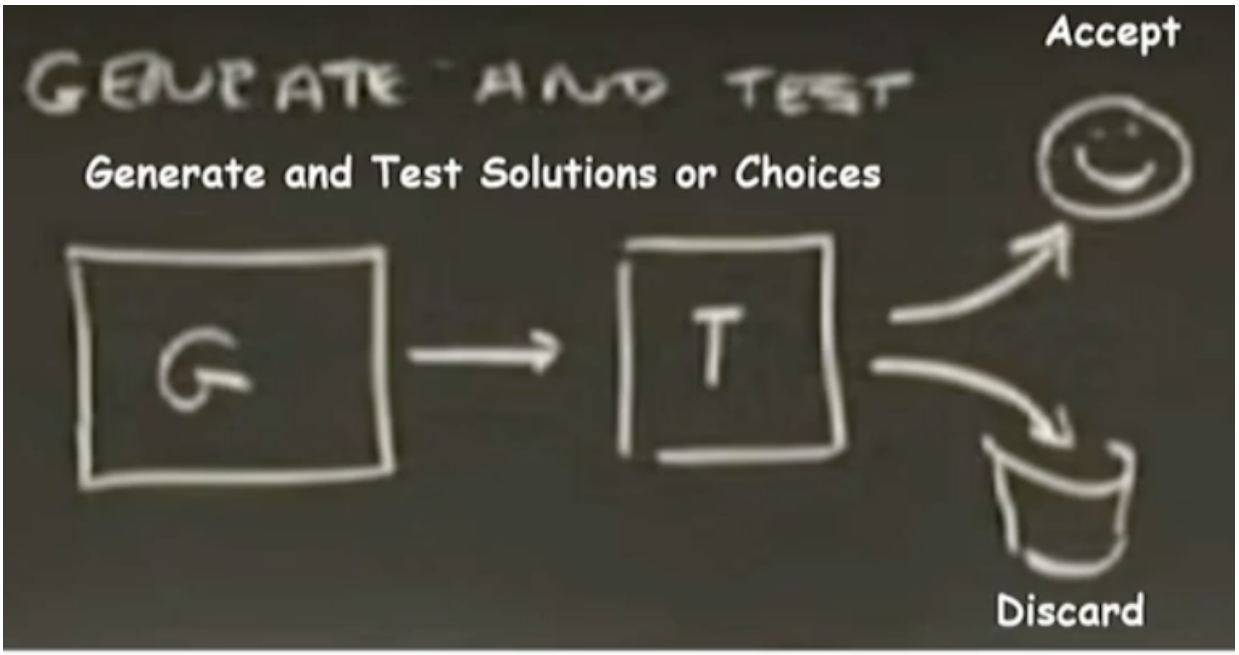
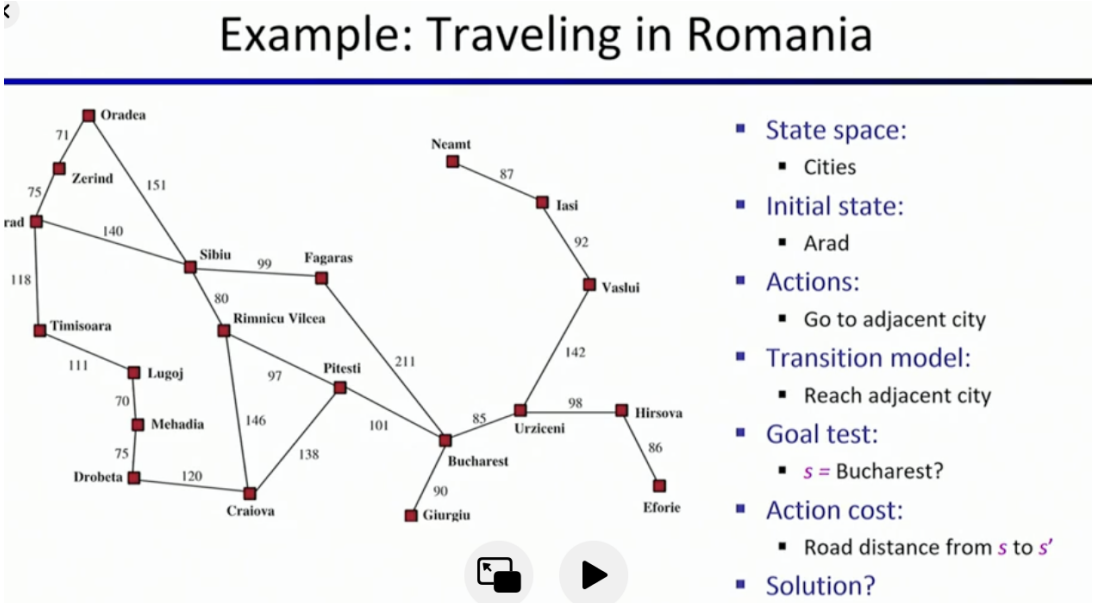
- Problema:
  - Vogliamo raggiungere delle condizioni che non sono ancora soddisfatte, e dobbiamo scegliere tra delle possibili opzioni
- Il risulatore generico:
  - 

    - Problema: troppo bianco e nero, ed è difficile da testare cose difficili, e da generare è troppo difficile siccome potremmo essere ridontanti
    - E da qui nascono i problemi di ricrca
- Problemi di ricerca
  - So lo stato iniziale, so lo stato finale, e il problema può essere risolto con una sequenza di passi
    - Modelli di transizioni ci dice, dato uno stato e un azione il nuovo stato
    - Noi abbiamo stati, e o ci siamo oppure non ci siamo
  - E potremmo anche avere delle condizioni per la soluzioni (esempio la soluzione migliore)
  - Esempio: google maps, e i constrains sono magari a passi, o macchina, e deve essere più breve
  - Definito formalmente:
    - Un problema di ricerca consiste in
      - Uno spazio di stati (tutti i possibili stati unici) = S
      - Lo stato iniziale = s_0
      - Tutte le possibili azioni per ogni stato = $A(s)$
      - Funzione successore = $s' = next(s, a)$
      - Il nostro goal $G(s)$

    - Costo di ogni azione $c(s, a, s')$
- la soluzione è una sequenza di azione che già porta a $G\,(s)$
    - Per trovare la soluzione, dovremo scrivere un insieme di soluzioni parziali
    - E siccome le soluzioni parziali sono in un albero, una soluzione parziale è il percorso da un nodo verso l'origine
- E la soluzione ottimale è una soluzione che ha il minor numero di costo
- E negli esercizi dobbiamo sempre specificare ognuno di essi
    - 

- 1 problema: lo spazio degli stati deve essere compatta
    - world state è troppo grande, dobbiamo ottenere una search state. Immagina pacman
        - Troppo dettagliato richiederà troppo tempo e non ci sta in memoria
        - Troppo poco dettagliato e taglia possibili soluzioni
    - Se il nostro goal è quello di raggiungere bucharest e dobbiamo evitare di passare 3 citta consecutive con I nel loro nome, lo stato e world state aumenta veramente tanto
    - Nell'esempio di sopra, di arrivare a bucarest da arad, partiamo da arad e cerchiamo in tutte le città adiacenti
      - Potremmo entrare in un loop

- Potremmo trovare non la soluzione migliore

```
PartialSolutionSet =[Initial_State]
While not(PartialSolutionSet.empty()):
    solution=extractSolution(PartialSolutionSet)
    new_state=getState(solution)
    Successors=[]
    for action in actions_valid_in(new_state):
        Successors.append(successor(new_state,action)
    for state in Successors:
      new_solution=solution.copy().append(state)
      if goalconditionn(new_state) then:
        return "victory"
      PartialSolutionSet.insert(new_solution)
```

  - Tutte le parti tranne dove sottolineato, sono obbligatorie, e noi possiamo solo agire in excactSolution e insert, li sono gli unici punti in cui possiamo agire a seconda dei constraint
  - extractSolution ed insert sono il nostro generatori
  - **Importante**: tutti gli algoritmi di ricerca si differiscono da solo queste 2 parti
  - Questa rappresentazione **è un albero**
  - Quindi abbiamo un search tree, ad ogni nodo espandiamo sotto di essa, e la soluzione è un percorso che parte dalla radice alla foglia
  - Un pericolo è che certi grafi hanno un percorso infinito per via dei loop
- Algoritmi:
  - Backtracking search
    -

Algorithm: backtracking search

```
def backtrackingSearch(s, path, D):
    if length(path)>D: return NULL
    If IsEnd(s): update minimum cost path
    For each action a ∈ Actions(s):
        Extend path with Succ(s, a)
        Call backtrackingSearch(Succ(s, a), path,D)
    Return minimum cost path
```

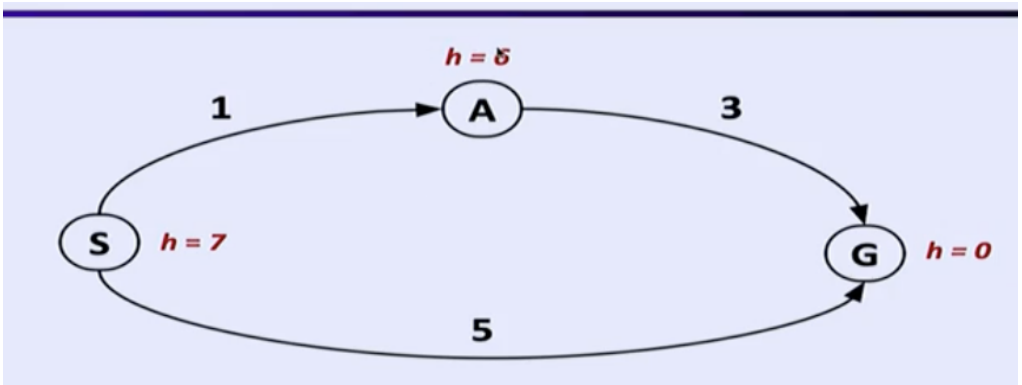
  - Ciò che facciamo è praticamente una ricerca tutto a sinistra, e quando arrivata una foglia, facciamo 1 passo indietro, poi a destra e ricominciamo
    - Tutto a sinistra, backtracking, e poi 1 destra, e si continua finchè non abbiamo esplorato tutto l'albero
    - Quindi espande tutti i nodi
  - Problema: potrebbe essere che incombiamo nell'infinito
    - D è la nostra profondità massima, nota che se non troviamo la soluzione in D dovremo ricominciare da capo ed aumentare D

        - Questo è opzionale, serve per cercare un middleground col BFS
    - Il vantaggio è che dobbiamo tenere in memoria solo l'ultimo nodo con soluzione migliore
    - Lo svantaggio è che se la soluzione è tutto a destra, dobbiamo esplorare l'intero albero
- Un esempio di questo algoritmo è il **Depth First Search**
    - Spazio: O(azioni * massima profondità)
        - Siccome dobbiamo salvarsi il percorso
        - E lo gestisce come LIFO (ricorsione)
    - Tempo: O(azioni^massima profondità)
        - Siccome potremmo dover esplorare tutti i percorsi
        - Espande tutti i nodi ed i suoi successori
    - Rispetto al BFS, trova il percorso minimo più propenso rispetto al costo
    - Il DFS si interrompe appena trova 1 soluzione, come il BFS
- Algoritmo per profondità BFS
    - Qui invece andiamo per layer a layer, lo avevamo fatto in altri corsi
    - Qui non possiamo fare una ricorsione ed è FIFO come gestisce
        - BFS e DFS differiscono solo su questo
    - Lo spazio qui è uguale al tempo che è O(azione^massima profondità)
    - Questo algoritmo trova il percorso minore in termini di azioni, ma non in termini di costo
    -
```
PartialSolutionSet =[Initial_State]
While not(PartialSolutionSet.empty()):
    solution=head(PartialSolutionSet)
    if len(solution)>D then return "fail"
    new_state=getState(solution)
    Successors=[]
    for action in actions_valid_in(new_state):
        Successors.append(successor(new_state,action)
    for state in Successors:
        new_solution=solution.copy().append(state)
        if goalconditionn(new_state) then:
            return "victory",new_solution
        PartialSolutionSet.append(new_solution)
return "fail"
```
- Uniform cost search
    - Espandiamo il percorso con il costo più piccolo
    - Uguale a BFS, ma anzichè percorso minimo andiamo costo minimo

```
PartialSolutionSet =[Initial_State]
While not(PartialSolutionSet.empty()):
    solution=head(PartialSolutionSet)
    new_state=getState(solution)
    Successors=[]
    for action in actions_valid_in(new_state):
        Successors.append(successor(new_state,action)
    for state in Successors:
        new_solution=solution.copy().append(state)
        if goalconditionn(new_state) then:
            return "victory",new_solution
        PartialSolutionSet.append(new_solution)
        PartialSolutionSet.sort()
```

- Greedy search
  - Scegliamo il figlio con il costo migliore
- A* search
  - Combina UCS e greedy
  - Scegliamo il nodo con costo + euristica migliore: g(n)+h^(n)
    - G(n) costo dalla radice minore
    - H^(n) è una stima di quanto ci mettiamo ad arrivare all'obiettivo, un euristica
      - Una funzione euristica è una sottostima che prende punto e goal
    - Nota bene: $0 \quad (n) \ ^*(n)$
      - Con questo intendo che la stima deve essere più piccola del valore vero
      - Esempio la linea diretta tra 2 città è sempre più corta della strada tra 2 città
    - L'euristica deve **sempre** essere una sottostima

      -



Qui ad esempio, h(s)=7, però s->g=5= $^*(s)$, (s) $^*(s)$

